

Prueba técnica – Full Stack Developer (Web + Mobile Híbrido)



Objetivo

Construir una solución end-to-end para la gestión de tareas personales, compuesta por:

- Aplicación web SPA.
- Aplicación móvil híbrida con Ionic.
- API backend segura.
- Persistencia de datos.
- Documentación y despliegue local reproducible.

La prueba busca evaluar capacidad de diseño, desarrollo, integración, calidad de código y criterio técnico.

Contexto funcional

Se requiere una plataforma llamada TaskFlow para que un usuario autenticado pueda administrar sus tareas.

El sistema debe permitir crear, consultar, actualizar, completar y eliminar tareas. La información debe poder gestionarse tanto desde la aplicación web como desde la aplicación móvil.

Cada usuario solo puede consultar y administrar sus propias tareas.

Tecnologías requeridas

Frontend web

Usar una de las siguientes alternativas:

- Angular.
- React.

Debe ser una SPA e incluir manejo de estado mediante Redux, NgRx, Context + reducer, Zustand o una alternativa equivalente y justificada.

Aplicación móvil

- Ionic.
- Preferiblemente Ionic con Angular.
- Capacitor para ejecutar en Android o iOS.

La aplicación debe ser posible ejecutarla en un dispositivo o emulador.

Backend

- NestJS con Node.js.
- Java con Spring Boot es aceptado como alternativa.

Persistencia

Puede utilizar una de estas opciones:

- PostgreSQL.
- MySQL.
- MongoDB.
- SQLite, únicamente si deja clara la estrategia de migración para un ambiente productivo.

Funcionalidades mínimas

1. Autenticación

Implementar autenticación basada en JWT.

Debe incluir:

- Registro de usuario.
- Inicio de sesión.
- Protección de rutas o endpoints.
- Persistencia segura de contraseña mediante hash; por ejemplo, bcrypt.
- Cierre de sesión en web y mobile.
- Manejo básico de sesión expirada.

Campos mínimos del usuario:

- Nombre.
- Correo electrónico.
- Contraseña.

2. Gestión de tareas

Cada tarea debe contener como mínimo:

- Identificador.
- Título.
- Descripción.
- Estado: pendiente o completada.
- Prioridad: baja, media o alta.
- Fecha límite.
- Fecha de creación.
- Usuario propietario.

Operaciones requeridas:

- Crear tarea.
- Consultar listado de tareas propias.
- Consultar detalle de una tarea.
- Editar tarea.
- Marcar una tarea como completada o pendiente.
- Eliminar tarea.
- Filtrar por estado y prioridad.
- Buscar por título.

3. Aplicación web

La aplicación web debe contar como mínimo con:

- Pantalla de registro e inicio de sesión.
- Vista principal con listado de tareas.
- Formulario para crear y editar tareas.
- Filtros y buscador.
- Indicadores visuales de prioridad y estado.
- Estados de carga, vacío y error.
- Diseño responsive básico.
- Protección de rutas privadas.

Se valorará especialmente:

- Buena organización de componentes.
- Manejo claro del estado.
- Validación de formularios.
- UX clara y consistente.
- Rendimiento razonable y evitar renderizados innecesarios.

4. Aplicación móvil híbrida

La aplicación móvil debe permitir, como mínimo:

- Inicio de sesión.
- Consulta del listado de tareas.
- Crear una tarea.
- Marcar una tarea como completada.
- Aplicar filtros básicos.
- Cerrar sesión.

Además, debe integrar al menos una capacidad nativa mediante Capacitor. Ejemplos aceptados:

- Notificación local al crear una tarea de alta prioridad.
- Uso de cámara para adjuntar una imagen opcional a una tarea.
- Uso de biometría para desbloquear la sesión.
- Manejo de conectividad para informar al usuario si está sin internet.

No se espera una implementación de todas las capacidades nativas; se evaluará la correcta integración de una de ellas.

5. API backend

La API debe implementar, como mínimo, los siguientes recursos:

- POST /auth/register
- POST /auth/login
- GET /tasks
- POST /tasks
- GET /tasks/:id
- PATCH /tasks/:id
- DELETE /tasks/:id

Requisitos técnicos:

- API REST.
- DTOs o contratos de entrada/salida.
- Validación de datos.
- Manejo centralizado de errores.
- Autorización: un usuario no debe poder acceder a las tareas de otro.
- Documentación con Swagger/OpenAPI.
- Variables de entorno mediante archivo .env.example.

Requisitos de calidad y arquitectura

Se espera una estructura clara y mantenible, se debe aplicar principios de:

- Separación de responsabilidades.
- SOLID, cuando sea aplicable.
- Componentes, servicios y módulos con responsabilidades definidas.
- Código reutilizable.
- Manejo de errores.
- Nombres claros y consistentes.
- No incluir secretos, tokens ni credenciales en el repositorio.

Deseables

No son obligatorios, pero suman en la evaluación:

- Docker y Docker Compose para levantar backend y base de datos.
- Pipeline CI/CD con GitHub Actions.
- Pruebas unitarias en frontend.
- Pruebas e2e con Cypress, Playwright o similar.
- Paginación en listado de tareas.
- Arquitectura modular o basada en dominios.

- Refresh token.
- Control de roles.
- Auditoría básica de acciones.
- Manejo offline o almacenamiento local en mobile.
- Accesibilidad.
- Despliegue en cloud.

Entregables

Se debe crear un repositorio privado o público en GitHub y compartir el enlace.

La estructura puede ser monorepo o repositorios separados, pero se recomienda la siguiente:

```
taskflow/
├── backend/
├── web/
├── mobile/
├── docker-compose.yml
├── README.md
└── .env.example
```

El repositorio debe incluir:

1. Código fuente de backend, web y mobile.
2. Archivo README.md con:
 - o Descripción de la solución.
 - o Arquitectura propuesta.
 - o Tecnologías utilizadas y justificación.
 - o Instrucciones detalladas para ejecutar cada componente.
 - o Variables de entorno requeridas.
 - o Decisiones técnicas relevantes.
 - o Evidencia de pruebas ejecutadas.
 - o Capturas de pantalla de la web y de la aplicación móvil.
 - o Limitaciones conocidas y mejoras futuras.
3. Archivo .env.example.
4. Explicación breve de la arquitectura.